

file objects. It has a pointer *fd* to each opened file, which is defined by the *file* structure. This *file* structure has a pointer *f\_ops*, which points to the file operation handlers or routines to operate file operations like read, write, open and close.

## Threads

Linux does not have threads. Linux treats threads as processes, which means that threads are processes that share the memory address space. If a process has two threads, it means two processes are sharing resources.

Threads are created by the clone system call with the following flags:

```
>> CLONE_VM: Parent and child share address space
>> CLONE_FS: Parent and child share file system info
>> CLONE_FILES: Parent and child share open files
>> CLONE_SIGHAND: Parent and child share signal handlers and
blocked signals
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);
```

For the process, it will be:

```
clone(SICHL, 0);
```

## Kernel threads

Linux also has kernel threads that are used for kernel related background operations because normal threads are switched between the kernel space and user space, and this is not safe for kernel tasks like flush tasks, *ksortiqd* tasks, etc. Kernel threads don't have address space, which means the *mm* pointer points to NULL.

Kernel threads are created by the *kthread\_create* API and declared in `<linux/kthread.h>`:-

```
struct task_struct *kthread_create(int (*threadfn)(void
*data), void *data, const char namefmt[], ...);
```

## Process states

Processes go through some states based on scheduling.

- **Running state:** In this state, the process is in a runnable state or in a run queue, waiting to run. The `TASK_RUNNING` flag is used for this state.
- **Interruptible state:** Here, the process is in sleep mode and waiting for some task to finish, but it can also be invoked prematurely by a signal. The `TASK_INTERRUPTIBLE` flag is used for this state.
- **Uninterruptible state:** The process is in sleep mode and waiting for some task to finish but it cannot be invoked prematurely by a signal. The `TASK_UNINTERRUPTIBLE` flag is used for this state.

So processes can be in the running, waiting or stopped state. There is one more state – the zombie state – which will be described later.

## Death of a process

It's nature's rule that no one can live forever. So, unfortunately, processes also have to die. A process dies by itself by calling *exit()* system call. This is how it self-destructs. Processes can also die from a received signal or exception, which they cannot handle. *Exit()* then calls *do\_exit()*, which calls *exit\_mm()*, *exit\_sem()*, *exit\_files()* and *exit\_fs()*, to release *mm\_struct*, to de-queue processes waiting for the semaphore, to decrement count of usage of file descriptors and file system data, respectively. It also calls *schedule()* so that the next process can be scheduled.

With this, all resources are released except the kernel stack, *task\_struct* and *thread\_info*. That means the process is in the zombie state. Now it's the responsibility of the parent process to inform the kernel that its child is not required; then the kernel removes the process completely by calling *release\_task()*, which again calls *put\_task\_struct()* to free the pages containing the process's kernel stack and *thread\_info* structure and de-allocate the slab cache containing the *task\_struct*. 

### By: Subodh Saxena

The author works in the device driver space. He is interested in new technologies, especially if they are open source. You can reach him at [subodhsaxena426@live.com](mailto:subodhsaxena426@live.com)

Read more stories on manufacturing at  
**www.electronicseb2b.com**

**TOP**  
MANUFACTURING  
STORIES

- Exclusive: ELCINA's first greenfield EMC project soon to get land possession
- India is emerging as an attractive destination for international companies to invest
- Electronics manufacturing industry in Pune needs a boost
- In 5 years, India may rank 2nd in manufacturing competitiveness, worldwide
- Highlights of state electronics policies
- National Policy on Electronics will Boost R & D in Electronic Manufacturing
- Insecure market threatens solar module makers

**ELECTRONICS**  
**B2B**  
INDUSTRY IS AT A

CLICK OF A BUTTON

Log on to [www.electronicseb2b.com](http://www.electronicseb2b.com) and be in touch with the Electronics B2B Fraternity 24x7